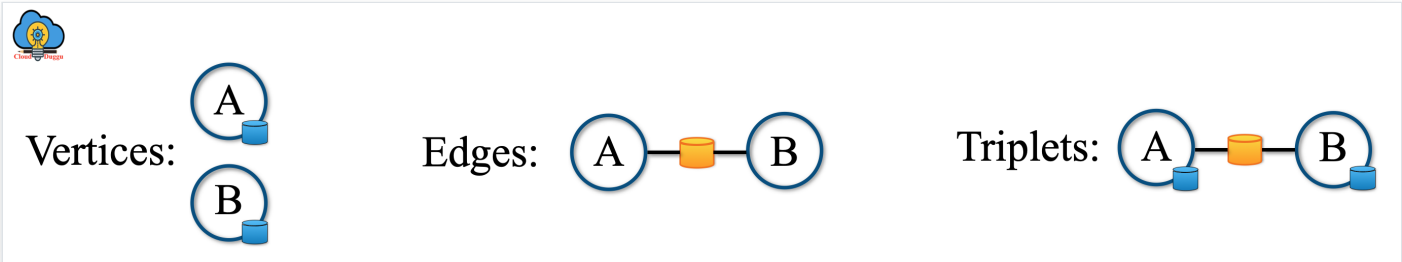




What is a Graph?

A Graph is a representation of the directed and undirected graph concept that is taken from mathematics graph theory. It is the abstract data type and a data structure that contains a measurable set of vertices with some unordered pair of vertices for an undirected graph and an ordered set of vertices for a directed set and together these sets are called edge.



What is Apache Spark GraphX?

Apache Spark GraphX is a distributed graph processing framework that is used to process graphs in parallel. It provides a collection of Graph algorithms and builders which are used to analyze the graph tasks easily. GraphX uses the Spark RDD to provides a new Graph abstraction. There is a property graph that has user-defined objects for each property graph.

Apache Spark GraphX Features

Apache Spark GraphX provides the following features.

1. Flexibility

Apache Spark GraphX is capable of working with graphs and perform the computations on them. Spark GraphX is be used for ETL processing, iterative graph computation, exploratory analysis, and so on. The data can be views as a collection as well as a graph and the transformation and joining of that data can be efficiently performed with Spark RDD.

2. Speed

Apache Spark GraphX provides better performance compared to the fastest graph systems and since it works with Spark so it by default adopts the features of Apache Spark such as fault tolerance, flexibility, ease to use, and so on.

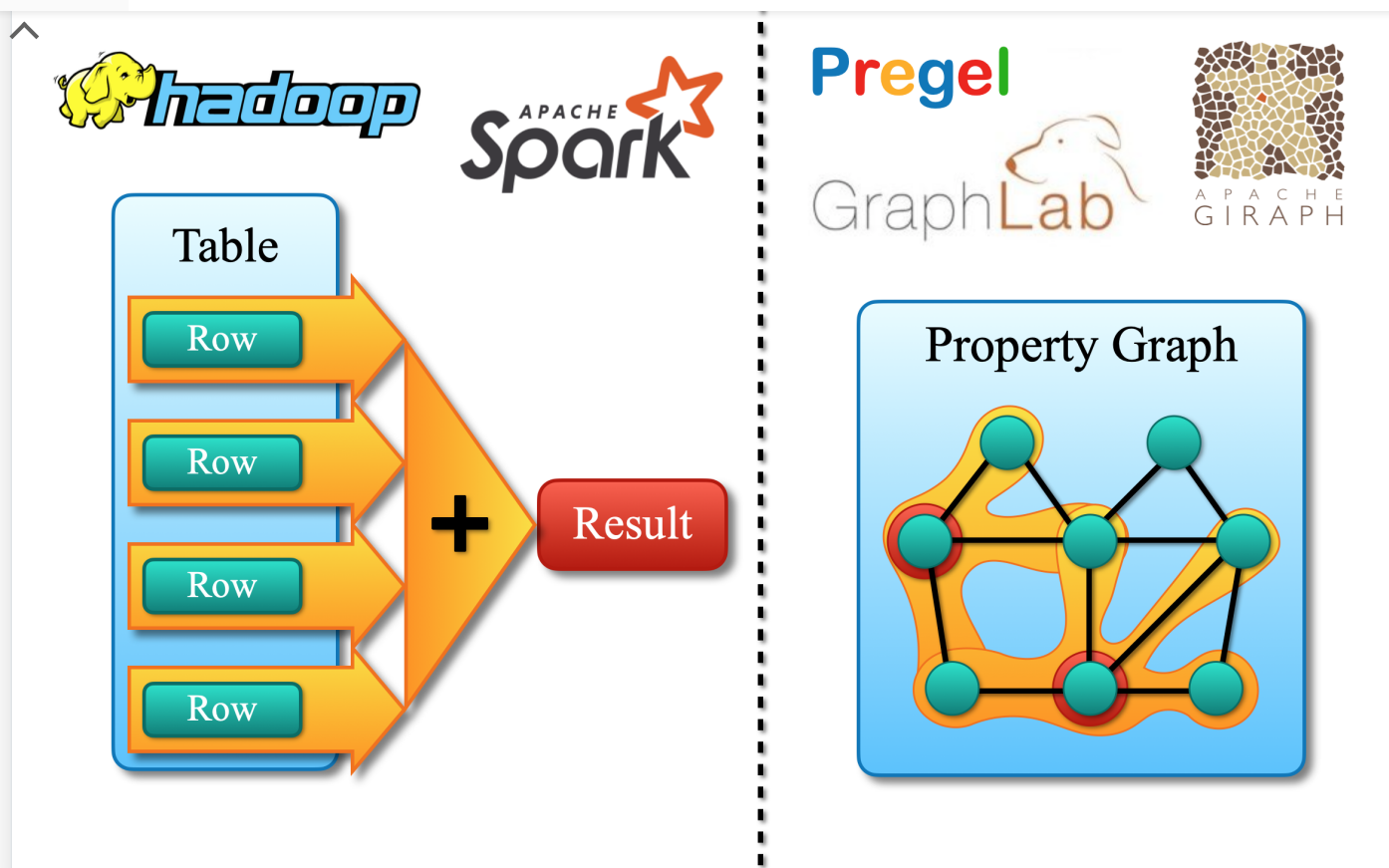
3. Algorithms

Apache Spark GraphX provides the following graph algorithms.

- PageRank
- Connected Components
- Label Propagation
- SVD++
- Strongly Connected Components
- Triangle Count

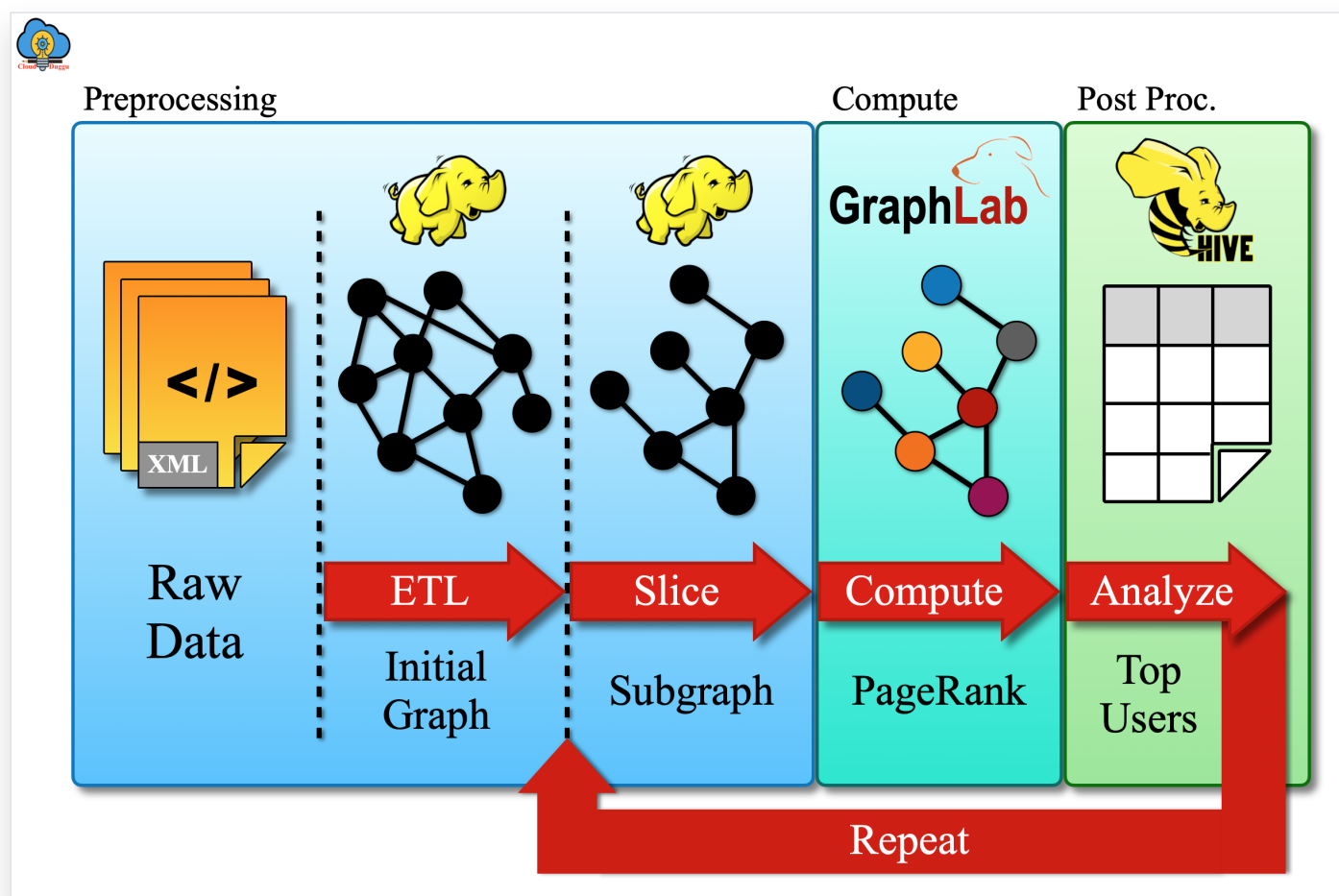
Background on Graph-Parallel Computation

The graph data keeps growing day after day and looking into its importance the new graph processing systems are getting developed. These systems are working on the new technology to perform partition and process graphs distributedly. The Graphics algorithms are getting tested with new systems to provides a faster response compared to the common data processing systems.



Apache Spark GraphX Project Goal

The existing Graph processing system work on the complicated programming model that leads to heavy data movement and duplications. So to overcome, this issue Apache Spark GraphX project is an ally to provide the data-parallel and graph-parallel processing to a single system with one composable API. Apache Spark GraphX API provides the facility to see data as a collection as well as graphs without any duplication or any data movement.



Graph Algorithms

Apache GraphX provides a set of graph algorithms to simplify analytics tasks. These algorithms are contained in the `org.apache.spark.graphx.lib` package and can be accessed directly as methods on Graph via GraphOps.

be ranked highly.

Let us understand the PageRank Algorithm with the help of the Social networking program.

PageRank Program Execution

We have two datasets one is for users and another one is for user's followers. We will compute the PageRank of each user as follows.

Datasets & Program Details

Datasets for this program are present under SPARK_HOME directory.

graphx/data/followers.txt

graphx/data/users.txt

Location of PageRankExample.scala program is mentioned below.

/home/cloudduggu/spark/examples/src/main/scala/org/apache/spark/examples/graphx/PageRankExample.scala

Run PageRankExample.scala program from SPARK_HOME.

./bin/run-example org.apache.spark.examples.graphx.PageRankExample

Let us understand code line by line and then we will execute it on terminal.

1. Import Spark's graphx and SQL package.
2. Creates a SparkSession.
3. Load the edges as a graph.
4. Run PageRank.
5. Join the ranks with the usernames.
6. Print the result.

```
import org.apache.spark.graphx.GraphLoader
import org.apache.spark.sql.SparkSession

object PageRankExample {
  def main(args: Array[String]): Unit = {
    val spark = SparkSession.builder
      .appName(s"${this.getClass.getSimpleName}")
      .getOrCreate()
    val sc = spark.sparkContext
    val graph = GraphLoader.edgeListFile(sc, "data/graphx/followers.txt")
    val ranks = graph.pageRank(0.0001).vertices
    val users = sc.textFile("data/graphx/users.txt").map { line =>
      val fields = line.split(",")
      (fields(0).toLong, fields(1))
    }
    val ranksByUsername = users.join(ranks).map {
      case (id, (username, rank)) => (username, rank)
    }
    println(ranksByUsername.collect().mkString("\n"))
    spark.stop()
  }
}
```



```
20/06/14 05:34:56 INFO DAGScheduler: ResultStage 4645 (collect at PageRankExample.scala:56) finished in 0.889 s
20/06/14 05:34:56 INFO DAGScheduler: Job 54 finished: collect at PageRankExample.scala:56, took 1.778486 s
(justinbieber,0.15007622780470478)
(matei_zaharia,0.7017164142469724)
(ladygaga,1.3907556008752426)
(BarackObama,1.4596227918476916)
(jeresig,0.9998520559494657)
(odersky,1.2979769092759237)
20/06/14 05:34:56 INFO SparkUI: Stopped Spark web UI at http://192.168.216.131:4041
20/06/14 05:35:02 INFO MapOutputTrackerMasterEndpoint: MapOutputTrackerMasterEndpoint stopped!
20/06/14 05:35:03 INFO MemoryStore: MemoryStore cleared
20/06/14 05:35:03 INFO BlockManager: BlockManager stopped
20/06/14 05:35:03 INFO BlockManagerMaster: BlockManagerMaster stopped
20/06/14 05:35:03 INFO OutputCommitCoordinator$OutputCommitCoordinatorEndpoint: OutputCommitCoordinator stopped!
20/06/14 05:35:08 INFO SparkContext: Successfully stopped SparkContext
20/06/14 05:35:08 INFO ShutdownHookManager: Shutdown hook called
20/06/14 05:35:08 INFO ShutdownHookManager: Deleting directory /tmp/spark-a497ef49-cbef-4224-9a7d-e9dc33546d73
20/06/14 05:35:08 INFO ShutdownHookManager: Deleting directory /tmp/spark-160f4555-87da-421e-9733-13a11df8c1f6
cLoudDuggu@ubuntu:~/spark$
```

2. Connected Components Algorithm

In the connected components algorithm, there is a subgraph in which each vertex is accessible to each other vertex by following edges. In this algorithm, every connected component of the graph is tagged with an ID from its lower vertex.

Connected Components Program Execution

We have two datasets one is for users and another one is for user’s followers. We will compute the PageRank of each user as follows.

Datasets & Program Details

Datasets for this program are present under SPARK_HOME directory.

graphx/data/followers.txt

graphx/data/users.txt

Location of ConnectedComponentsExample.scala program is mentioned below.

/home/cloudduggu/spark/examples/src/main/scala/org/apache/spark/examples/graphx/ConnectedComponentsExample.scala

Run ConnectedComponentsExample.scala program from SPARK_HOME.

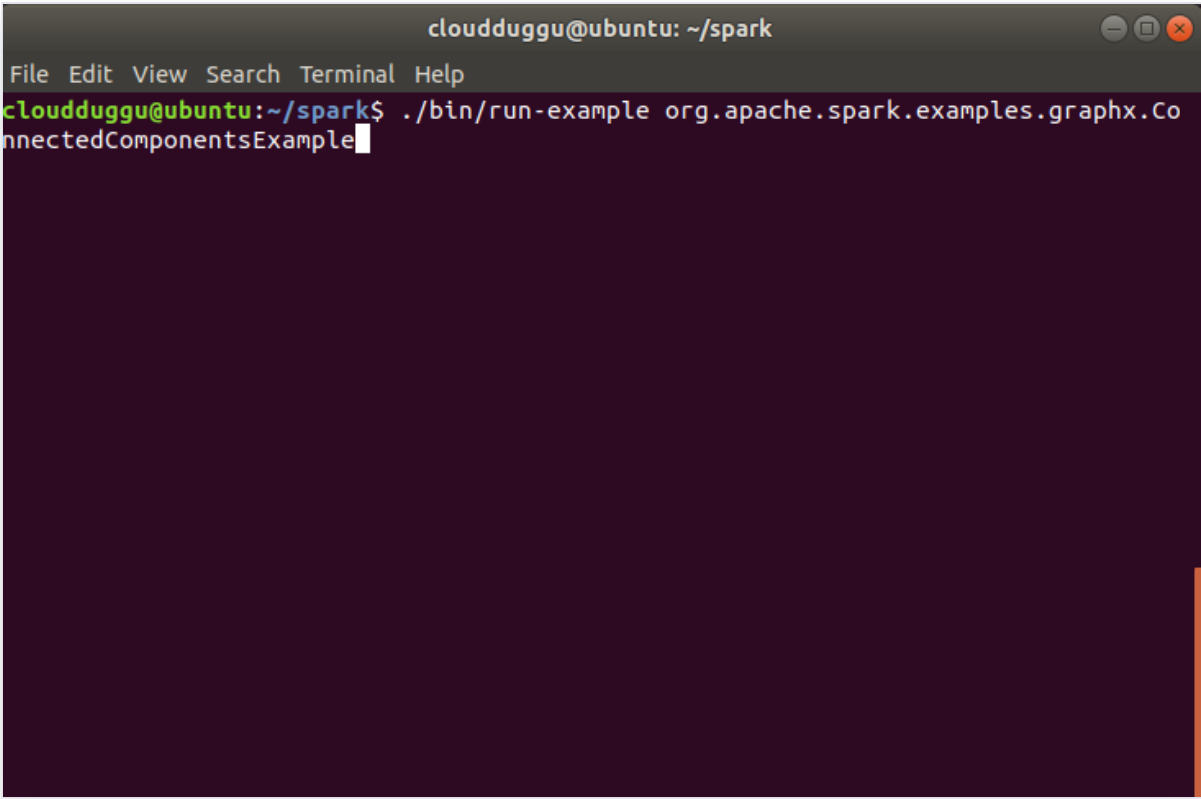
./bin/run-example org.apache.spark.examples.graphx.ConnectedComponentsExample

Let us understand code line by line and then we will execute it on terminal.

1. Import Spark's graphx and SQL package.
2. Creates a SparkSession.
3. Load the graph as in the PageRank example.
4. Find the connected components.
5. Join the connected components with the usernames.
6. Print the result.

```
import org.apache.spark.graphx.GraphLoader
import org.apache.spark.sql.SparkSession
```

```
        .getOrCreate()
    val sc = spark.sparkContext
    val graph = GraphLoader.edgeListFile(sc, "data/graphx/followers.txt")
    val cc = graph.connectedComponents().vertices
    val users = sc.textFile("data/graphx/users.txt").map { line =>
        val fields = line.split(",")
        (fields(0).toLong, fields(1))
    }
    val ccByUsername = users.join(cc).map {
        case (id, (username, cc)) => (username, cc)
    }
    println(ccByUsername.collect().mkString("\n"))
    spark.stop()
}
}
```



```
20/06/14 06:42:19 INFO DAGScheduler: ResultStage 19 (collect at ConnectedComponentsExample.scala:63) finished in 0.527 s
20/06/14 06:42:19 INFO DAGScheduler: Job 3 finished: collect at ConnectedComponentsExample.scala:63, took 1.145206 s
(justinbieber,1)
(matei_zaharia,3)
(ladygaga,1)
(BarackObama,1)
(jeresig,3)
(odersky,3)
20/06/14 06:42:19 INFO SparkUI: Stopped Spark web UI at http://192.168.216.131:4041
20/06/14 06:42:19 INFO MapOutputTrackerMasterEndpoint: MapOutputTrackerMasterEndpoint stopped!
20/06/14 06:42:19 INFO MemoryStore: MemoryStore cleared
20/06/14 06:42:19 INFO BlockManager: BlockManager stopped
20/06/14 06:42:19 INFO BlockManagerMaster: BlockManagerMaster stopped
20/06/14 06:42:19 INFO OutputCommitCoordinator$OutputCommitCoordinatorEndpoint: OutputCommitCoordinator stopped!
20/06/14 06:42:19 INFO SparkContext: Successfully stopped SparkContext
20/06/14 06:42:19 INFO ShutdownHookManager: Shutdown hook called
20/06/14 06:42:19 INFO ShutdownHookManager: Deleting directory /tmp/spark-fe78f433-9566-413c-9166-c7c78581ed26
20/06/14 06:42:19 INFO ShutdownHookManager: Deleting directory /tmp/spark-2ecf1abb-5584-4a3f-89ca-3c44f634e200
cloudduggu@ubuntu:~/spark$
```

3. Triangle Counting Algorithm

The Triangle Counting Algorithm provides the two nearby vertices and an edge in between them that present vertex.

Triangle Counting Program Execution

We have two datasets one is for users and another one is for user’s followers. We will compute the PageRank of each user as follows.

Datasets & Program Details

Datasets for this program are present under SPARK_HOME directory.

graphx/data/followers.txt

graphx/data/users.txt

Run TriangleCountingExample.scala program from SPARK_HOME.

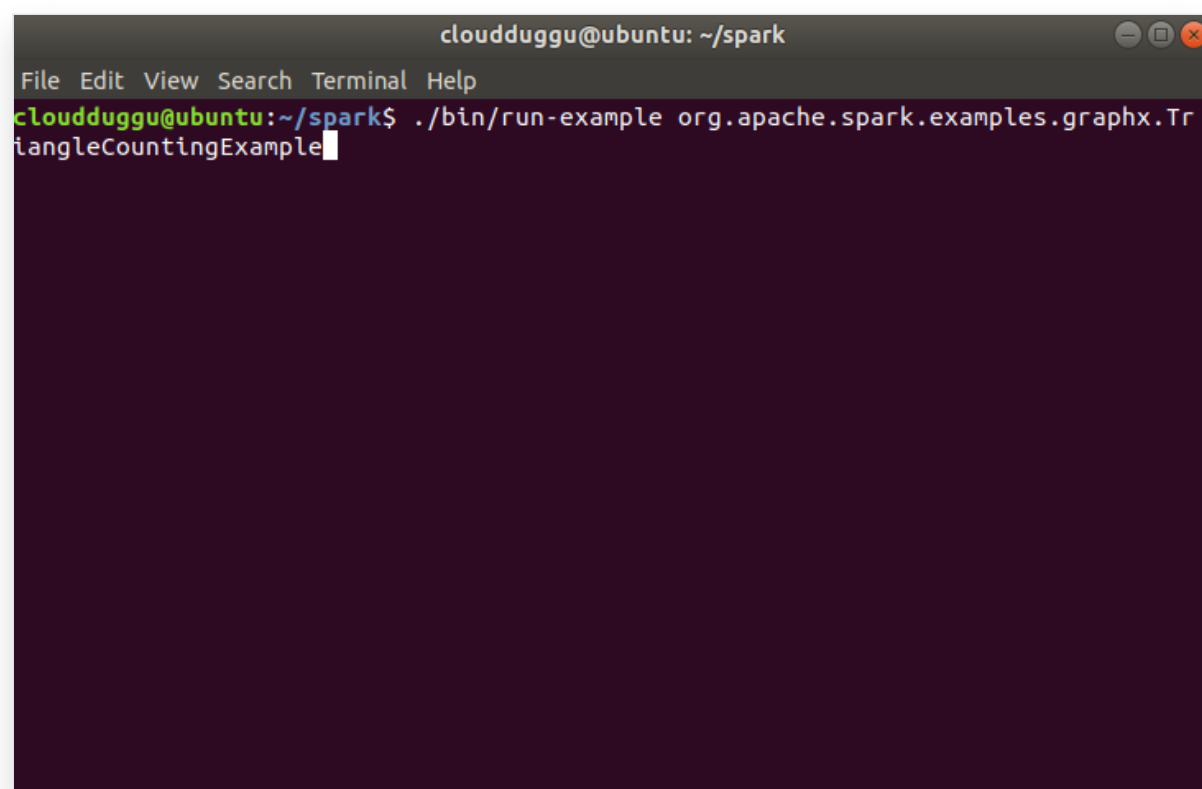
`./bin/run-example org.apache.spark.examples.graphx.TriangleCountingExample`

Let us understand code line by line and then we will execute it on terminal.

1. Import Spark's graphx and SQL package.
2. Creates a SparkSession.
3. Arrange the edge in the order of canonical and perform the graph partition to count the triangle.
4. Find the triangle count for each vertex.
5. Join the triangle counts with the usernames.
6. Print the result.

```
import org.apache.spark.graphx.{GraphLoader, PartitionStrategy}
import org.apache.spark.sql.SparkSession

object TriangleCountingExample {
  def main(args: Array[String]): Unit = {
    val spark = SparkSession.builder
      .appName(s"${this.getClass.getSimpleName}")
      .getOrCreate()
    val sc = spark.sparkContext
    val graph = GraphLoader.edgeListFile(sc, "data/graphx/followers.txt", true)
    .partitionBy(PartitionStrategy.RandomVertexCut)
    val triCounts = graph.triangleCount().vertices
    val users = sc.textFile("data/graphx/users.txt").map { line =>
      val fields = line.split(",")
      (fields(0).toLong, fields(1))
    }
    val triCountByUsername = users.join(triCounts).map { case (id, (username, tc)) =>
      (username, tc)
    }
    println(triCountByUsername.collect().mkString("\n"))
    spark.stop()
  }
}
```



⬆

```
07:08:41 INFO SparkUI: Stopped Spark Web UI at http://192.168.216.131:4042
07:08:41 INFO BlockManagerInfo: Removed broadcast_12_piece0 on 192.168.216.131:40221 in memory (size: 3.5 KB, free: 413.8 MB)
07:08:41 INFO MapOutputTrackerMasterEndpoint: MapOutputTrackerMasterEndpoint stopped!
20/06/14 07:08:41 INFO MemoryStore: MemoryStore cleared
20/06/14 07:08:41 INFO BlockManager: BlockManager stopped
20/06/14 07:08:41 INFO BlockManagerMaster: BlockManagerMaster stopped
20/06/14 07:08:41 INFO OutputCommitCoordinator$OutputCommitCoordinatorEndpoint: OutputCommitCoordinator stopped!
20/06/14 07:08:41 INFO SparkContext: Successfully stopped SparkContext
20/06/14 07:08:41 INFO ShutdownHookManager: Shutdown hook called
20/06/14 07:08:41 INFO ShutdownHookManager: Deleting directory /tmp/spark-81d7102c-dcb7-463d-936d-6b5892b7a396
20/06/14 07:08:41 INFO ShutdownHookManager: Deleting directory /tmp/spark-6f993840-c191-4e27-a0f6-830e1c7f13fd
cloudDuggu@ubuntu:~/spark$
```

Free Download

Free Forever. No Hidden Fees.

✕ ⓘ

W



© 2023 Copyright: [CloudDuggu.com](https://www.cloudduggu.com)